

PyQCU 全库结构分析

Lattice QCD 的 Python/Cython 库：CUDA 加速 Dirac 算子与多重网格求解器

opencode analy

2026-08-18

摘要

本报告对 PyQCU 仓库（/root/PyQCU）进行证据驱动的全库结构分析，覆盖两层执行层级、三张扁平张量桥接协议、调用生命周期不变量、张量布局约定、多线程多卡 MultiGrid 路径、h5py 持久化体系与测试体系。三视角结论：主体结构呈「纯 Python 层 Cython 桥 C++ CUDA 后端」三层；各部分以参数协议 `params/argv/set_ptrs` 与 `_SET_PLAN_` 计划选择织成依赖网；项目思路是用双路径（参考实现 + 生产后端）兼顾正确性与性能，并以一线程一卡并行突破单机多卡瓶颈。分析基于实测文件统计与逐文件行号证据，覆盖表闭合。

目录

1	仓库概览	1
1.1	规模统计与覆盖	1
2	主体结构	2
3	各部分关系	2
4	项目思路	3
4.1	设计意图	3
4.2	演进脉络	3
4.3	典型工作流	4
5	主题分析	4
5.1	两层执行层级架构	4
5.2	参数协议与调用生命周期	4
5.3	<code>_SET_PLAN_</code> 计划选择	4
5.4	张量布局约定	5
5.5	多线程多卡 MultiGpuMultigrid	5
5.6	h5py 持久化体系	5
5.7	测试体系	5

6 关键代码/文档片段

5

7 参考源清单

6

8 结论与局限

6

1 仓库概览

PyQCU 是 Lattice QCD 的 Python/Cython 库: CUDA 加速的 Wilson/Clover Dirac 算子、BiStabCG 与多重网格 (MultiGrid) 求解器、stout smearing 与规范场生成, 经 MPI 分布于 4D 进程网格 (AGENTS.md:1-10)。git 元信息: 分支 main, HEAD = a456932, 标签体系 stab<N>/dev<N>/bug<N>, 初始提交 0204fa5 (实测 git rev-parse)。

1.1 规模统计与覆盖

下表为排除 .git/、.opencode/、外部镜像 refer/git-rep/、logs/ 历史产物与 pyqcu/testing/ 日志后的实测统计 (子代理 find 实测)。

指标	实测值
Python (.py)	121
CUDA 内核 (.cu)	30
C++ 头 (.h)	24
文档 (.md)	45
仓库总文件 (排除噪声)	325
标签数	132

表 1: 仓库实测统计 (数据来源: find/wc/git 实测, 2026-08-18)

饱和覆盖登记表 (四条遍历路径交叉: 目录/类型/git 历史/入口引用链):

目录	文件数	状态与职责
pyqcu/	26 .py + 13 .md	精读: 纯 Python 算子/求解器/smear/lattice/cuda 桥/工具/testing
cpp/cuda/qcu/src/	30 .cu	精读: CUDA 内核 + 启动封装 (22 个 C API 对应)
cpp/cuda/qcu/include/	23 .h	精读: 模板化内核头 (含 define.h)
cpp/cuda/qcu/python/	1 .h	精读: C API pyqcu.h (须与 pxd 同步)
cpp/{cann,dtk,maca}/	占位	仅索引: PASS stub, 无实现
pyqcu/testing/	95 .py + 11 .md	浏览: 33 个 conftest 入口 + 后端测试脚本
docs/	7 .md	浏览: 维度/环境/安装/示例/剖析说明

表 2: 饱和覆盖登记表 (数据来源: 全库索引实测)

2 主体结构

PyQCU 呈清晰的三层分层：纯 Python 参考实现层、Cython 桥接层、C++ CUDA 生产后端层，外加兼容层与文档/产物层。

层次	目录	职责
入口层	pyqcu/、pyqcu/testing/、docs/、logs/	库入口、测试入口、文档、产物归档
核心层	pyqcu/dslash/、pyqcu/solver/、pyqcu/smear/	纯 Python 算子/求解器/smear (PyTorch)
桥接层	pyqcu/cuda/	Cython 桥 qcu.pyx + 参数常量 define.py + 多线程多卡
后端层	cpp/cuda/qcu/	26 个模板头 + 30 个 .cu + C API pyqcu.h
兼容层	pyqcu/cann/、cpp/{cann, dtk, maca}/	昇腾 NPU 兼容 (实) / 其他后端占位 (PASS)
参考层	refer/	外部库镜像 (DDalphaAMG/PyQUDA/quda) , 参考背景

表 3: 主体结构分层 (数据来源: 全库索引实测 + AGENTS.md:14-17)

两层执行层级是设计主轴 (AGENTS.md:14-17) : 纯 Python 层 (pyqcu/dslash/、solver/、smear/) 为 PyTorch 实现, 可跑 CPU/CUDA/昇腾 NPU (经 pyqcu.cann 兼容层); C++ CUDA 后端 (cpp/cuda/qcu/) 为手写内核 + MPI halo 交换, 经 Cython 桥暴露, 是生产路径。

3 各部分关系

依赖主链为「纯 Python 层 → Cython 桥 → C++ CUDA 后端」, 由参数协议 params/argv/set_ptrs 串联 (AGENTS.md:21)。

要点

依赖主链: pyqcu/dslash/_wilson.py → pyqcu/cuda/qcu/qcu.pyx → cpp/cuda/qcu/src/wilson_dslash.cu → cpp/cuda/qcu/include/lattice_wilson_dslash.h

- **协议同步链:** pyqcu/cuda/define.py:11-90 与 cpp/cuda/qcu/include/define.h:43-122 一一对应 (_LAT_X=0、_SET_INDEX=15、_SET_PLAN=16 等) , 任一失配导致静默内存损坏 (AGENTS.md:21)。
- **桥接别名链:** qcu.pyx:1 from qcu_api cimport X as _c_X 避免与 pyx def 同名覆盖, qcu_api.pxd:1-23 全部带 nogil (子代理核实)。

- **混合求解链**: 多重网格可混合两层——最细层平滑用 C++ 后端 (`with_cuda_qcu=True`), 更粗层用纯 Python (`AGENTS.md:28` 隐含)。
- **持久化链**: `pyqcu/tools/_io.py` 的 `save_tensor_h5/load_tensor_h5` 与 MPI `mpio` 路径, 被 `_multi_gpu.py:139-143` 的粗算子缓存复用。

4 项目思路

4.1 设计意图

双路径设计 (参考实现 + 生产后端) 兼顾**正确性** (Python 层可跨平台验证物理) 与**性能** (C++ CUDA 内核生产路径)。参数协议用三张扁平张量而非复杂对象, 便于 Python 与 C++ 零拷贝桥接 (`AGENTS.md:21`)。

4.2 演进脉络

从初始提交 `0204fa5` 到 `stab27` 共 1120 提交; 自有库代码主导文件为 `cpp/cuda/qcu/include/lattice_clover_multigrid.h` (+1833)、`cpp/cuda/qcu/src/clover_dslash_multi.cu` (+1610)、`cpp/cuda/qcu/src/wilson_dslash.cu` (+1442) 与 `pyqcu/tools/_multigrid.py` (+819)、`pyqcu/cuda/_multi_gpu.py` (+568), 显示项目从 Wilson dslash 演进到 Clover + 多重网格 + 多线程多卡 (`diff init→stab27` 实测)。

4.3 典型 workflow

一次典型任务: `source ./env.sh` → 构建 `libqcu.so` (`build.sh`) → 构建 Cython 扩展 (`install.sh`) → 在 `pyqcu/testing/python/conftest.py` 解锁测试 → 运行。生产调用生命周期: `applyInitQcu` → 操作 → `params[_SET_INDEX_] += 1` → `applyEndQcu` (`AGENTS.md:23`)。

项目思路

这个仓库之所以长成「纯 Python 参考 + C++ 生产 + Cython 薄桥 + 多线程多卡」的形态, 是为了在保持物理实现可验证的同时, 用 GPU 内核与一线程一卡并行逼近生产级格点 QCD 求解性能。

5 主题分析

5.1 两层执行层级架构

纯 Python 层约定所有代码 `import pyqcu.cann as _torch`, 不得直接 `import torch`——NPU 不支持复数张量, `cann` 层自动分解实虚部 (`AGENTS.md:16`)。C++ 后端经 `pyqcu/cuda/qcu/qcu.pyx` 暴露 (`AGENTS.md:17`)。

5.2 参数协议与调用生命周期

三个扁平张量桥接 Python C++ (pyqcu/cuda/define.py:8,82,90) : params(int32[54])、argv(float[7])、set_ptrs(int64[100])。关键不变量：连续 C++ 调用间必须 params[define._SET_INDEX_]+=1 (AGENTS.md:23)，否则 scratch 缓冲复用冲突、结果错误。

```

1 _DATA_TYPE_SIZE_ = 10
2 _SET_INDEX_ = 15
3 _SET_PLAN_ = 16
4 _SET_PLAN_N_2_ = -2 # just for laplacian
5 _SET_PLAN_N_1_ = -1 # just for gauss gauge
6 _SET_PLAN0_ = 0 # for wilson dslash
7 _SET_PLAN1_ = 1 # just for bistabcg and cg and the whole dslash for them
8 _SET_PLAN2_ = 2 # for clover dslash

```

5.3 __SET_PLAN__ 计划选择

define.py:38-43 给出算子类型：-2 Laplacian、-1 Gauss gauge、0 Wilson dslash、1 BiStabCG/CG、2 Clover dslash（子代理核实 define.py:38-43）。

5.4 张量布局约定

时空维永远是最后 4 轴 (...xyzt) , ward 索引用负整数 (wards['x']==-4) : 规范场 [3,3,4,Lx,Ly,Lz,Lt]、费米子场 [4,3,Lx,Ly,Lz,Lt]、Clover 项 [4,3,4,3,Lx,Ly,Lz,Lt] (AGENTS.md:35-37; pyqcu/lattice/__init__.py:16-20 定义 wards)。

5.5 多线程多卡 MultiGpuMultigrid

pyqcu/cuda/_multi_gpu.py 的 MultiGpuMultigrid 单进程内 N 线程 × 卡绑定并行；每线程独立 params/argv/set_ptrs 副本 (_SET_INDEX_ 各自从 0 计数)，要求单 MPI rank（子代理核实 _multi_gpu.py:234-242）。

```

1 class MultiGpuMultigrid(object):
2     """多线程多卡 C++ Clover Multigrid 驱动（一线程一卡）。
3
4     构造参数:
5         lat_size: [Lx, Ly, Lz, Lt]
6         mass / atol: 求解质量参数与容差
7         num_levels / dof_list / mg_grid / num_restart / coarse_max_iter /
8         coarse_tol_factor / nv_iters: 粗网格配置 (同 conftest.schur.multigrid)
9         nthreads: 线程数 (默认 = 设备数)

```

5.6 h5py 持久化体系

pyqcu/tools/_io.py 的 save_tensor_h5/load_tensor_h5 每调用独立 File 句柄、多线程安全（子代理核实 _io.py:165-194）；粗算子缓存 _multi_gpu.py:139-143 单句柄一次写全部 dataset，勿逐 dataset 覆盖重建 (AGENTS.md:28)。

5.7 测试体系

测试函数集中在 `pyqcu/testing/__init__.py` (`test_dslash_wilson:38`、`test_dslash_clover:202`、`test_solver:292`、`test_multigrid_multithread:576`、`test_multi_gpu_multigrid:620` 等), 各 `pyqcu/testing/*/conftest.py` 为 `pytest` 入口 (`AGENTS.md:26`)。

6 关键代码/文档片段

Wilson dslash 的 hopping 项 (纯 Python 实现, 夸克相邻格点跃迁):

```

1      # Term 1: (1 - _) U_{x,} src_{x+}
2      src_plus = _torch.roll(src, shifts=-1, dims=ward)
3      U_src_plus = _torch.einsum('Ccxyzt,scxyzt->sCxyzt', U_mu, src_plus)
4      term1 = _torch.einsum(
5          'Ss,sCxyzt->SCxyzt', I_minus_gamma[ward_key], U_src_plus)
6      # Term 2: (1 + _) U_{x-,}^† src_{x-}
7      src_minus = _torch.roll(src, shifts=1, dims=ward)
8      U_dag_minus = _torch.roll(U_dag_mu, shifts=1, dims=ward)
9      U_dag_src_minus = _torch.einsum(
10         'Ccxyzt,scxyzt->sCxyzt', U_dag_minus, src_minus)
11     term2 = _torch.einsum(
12         'Ss,sCxyzt->SCxyzt', I_plus_gamma[ward_key], U_dag_src_minus)
13     # Combine terms and subtract from dest
14     hopping = term1 + term2
15     dest -= _factor * hopping

```

Cython 桥 with `nogil` 真并行入口:

```

1  # 不再使用模块级共享 cdef 变量 (曾依赖 GIL 串行化, 见 2026-07-28 注释)。
2  # * 调用者须保证: 每线程独立 params/argv/set_ptrs 副本与设备绑定
3  # (见 pyqcu/cuda/_multi_gpu.py、pyqcu/cuda/_schur_op.py)。
4  def applyInitQcu(_set_ptrs, _params, _argv):
5      cdef long set_ptrs, params, argv
6      set_ptrs = _set_ptrs.contiguous().data_ptr()
7      params = _params.contiguous().data_ptr()

```

7 参考源清单

参考源	类型	内容/作用
AGENTS.md:14-17	仓库	两层执行层级定义
AGENTS.md:21	仓库	参数协议 <code>params/argv/set_ptrs</code> 与同步要求
AGENTS.md:23	仓库	调用生命周期与 <code>__SET_INDEX__</code> 递增不变量
AGENTS.md:35-37	仓库	张量布局约定
pyqcu/cuda/define.py:8-90	仓库	<code>params/argv/set_ptrs</code> 常量定义
pyqcu/cuda/define.py:36-43	仓库	<code>__SET_PLAN__</code> 取值含义
pyqcu/lattice/__init__.py:16-20	仓库	<code>wards</code> 负索引表

参考源	类型	内容/作用
pyqcu/cuda/_multi_gpu.py:234-242	仓库	单 MPI rank 强制
pyqcu/cuda/qcu/qcu.pyx:1,10-16	仓库	Cython 桥 nogil 调用
pyqcu/dslash/_wilson.py:49-63	仓库	Wilson hopping 项实现
cpp/cuda/qcu/include/define.h:43-122	仓库	C++ 端参数协议镜像
pyqcu/tools/_io.py:165-194	仓库	h5py 持久化 API
pyqcu/testing/__init__.py:16-620	仓库	集成测试套件

8 结论与局限

结论

结构：PyQCU 以「纯 Python 参考层 Cython 桥 C++ CUDA 生产层」三层组织，兼容层覆盖 NPU/占位后端。关系：两层由参数协议 `params/argv/set_ptrs` 与 `_SET_PLAN_` 计划选择织成依赖网，桥接别名与 `pxd/pyqcu.h` 同步是正确性的关键锁点。思路：双路径 + 一线程一卡并行，在可验证性与生产性能之间取平衡；多重网格（Galerkin + R3 fix + 5-stream）是核心特色。

参考背景

参考目录 `refer/`（1559 文件，外部库镜像）与 `docs/`（7 文件）在本分析中作为背景引入；`refer/` 提供 QUDA/PyQUDA 对照，但非 PyQCU 自有代码。

局限与风险

未覆盖/局限：未验证 `cpp/cuda/qcu/src/` 全部 30 个内核的执行细节（仅抽样精读代表文件）；`pyqcu/testing/` 95 个测试脚本为浏览态，未逐文件解析；多线程多卡的实际性能数据未在本次静态分析中实测（见 diff 报告与 pure 报告）。